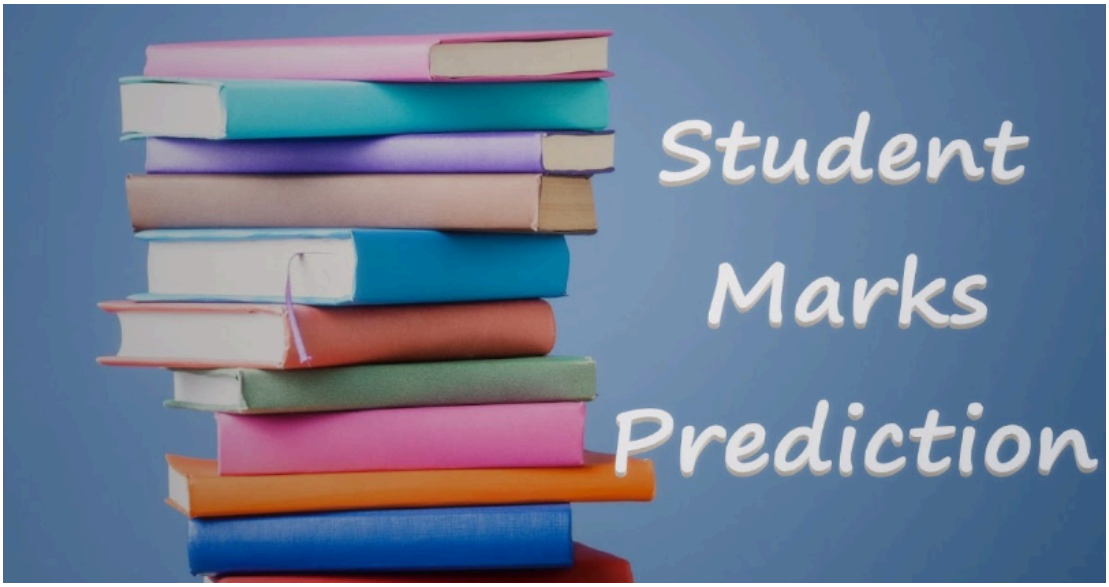
student mark predictor



In [1]:
```
#Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import  seaborn as sns
```

Load Dataset

In [2]:
```
marks = pd.read_csv(r"C:\Users\kunal\Downloads\student_info.csv")
```

In [3]:
```
marks
```

Out[3]:

	study_hours	student_marks
0	6.83	78.50
1	6.56	76.74
2	NaN	78.68
3	5.67	71.82
4	8.67	84.19
...	...	...
195	7.53	81.67
196	8.56	84.68
197	8.94	86.75
198	6.60	78.05
199	8.35	83.50

200 rows × 2 columns

Discover and Visualize the Data to Gain Insights (EDA)

- **Objective**

Exploratory Data Analysis (EDA) is the process of exploring, summarizing, and visualizing the dataset to understand its characteristics, identify patterns, detect missing values or outliers, and gain insights before building a Machine Learning model.

1. Display the First Few Records

In [4]:
```
marks.head()
```

Out[4]:

	study_hours	student_marks
0	6.83	78.50
1	6.56	76.74
2	NaN	78.68
3	5.67	71.82
4	8.67	84.19

- Displays the first 5 rows.
- Helps understand the dataset structure.

- Verifies whether the dataset loaded correctly.

2. Display the Last Few Records

In [5]:
```
marks.tail()
```

Out[5]:

	study_hours	student_marks
195	7.53	81.67
196	8.56	84.68
197	8.94	86.75
198	6.60	78.05
199	8.35	83.50

- Displays the last 5 rows.
- Confirms that the dataset was completely loaded.

3. Check Dataset Shape

In [6]:
```
marks.shape
```

Out[6]:
```
(200, 2)
```

- **200 Rows** → 200 students
- **2 Columns** → Study Hours and Student Marks

4. Display Column Names

In [7]:
```
marks.columns
```

Out[7]:
```
Index(['study_hours', 'student_marks'], dtype='object')
```

- Shows all column names.
- Helps identify input and target variables.

5.Dataset Information summary

In [8]:
```
marks.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 2 columns):
 #   Column         Non-Null Count  Dtype  
---  ------         --------------  -----  
 0   study_hours    195 non-null    float64
 1   student_marks  200 non-null    float64
dtypes: float64(2)
memory usage: 3.3 KB
```

The `info()` function provides a summary of the dataset, including the number of rows, columns, data types, non-null values, and memory usage.

Key Insights

- Total records: **200**
- Total columns: **2**
- `study_hours` has **195 non-null values** (5 missing values).
- `student_marks` has **200 non-null values** (no missing values).
- Both columns have the **float64** data type.
- Memory usage is approximately **3.3 KB**.

6 . Descriptive statistics

Statistical Summary

In [9]:
```
marks.describe()
```

Out[9]:

	study_hours	student_marks
count	195.000000	200.00000
mean	6.995949	77.93375
std	1.253060	4.92570
min	5.010000	68.57000
25%	5.775000	73.38500
50%	7.120000	77.71000
75%	8.085000	82.32000
max	8.990000	86.99000

In [10]:
```
marks.describe().T
```

Out[10]:

	count	mean	std	min	25%	50%	75%	max
study_hours	195.0	6.995949	1.25306	5.01	5.775	7.12	8.085	8.99
student_marks	200.0	77.933750	4.92570	68.57	73.385	77.71	82.320	86.99

The `describe()` function provides a summary of the numerical data in the dataset.

Explanation

- **Count** – Number of non-missing values.
- **Mean** – Average value of the column.
- **Std** – Standard deviation, showing how much the values vary from the average.
- **Min** – Smallest value in the column.
- **25%** – 25% of the values are below this value.
- **50% (Median)** – Middle value of the dataset.
- **75%** – 75% of the values are below this value.
- **Max** – Largest value in the column.

Key Insights

- The average study time is **7 hours**.
- The average student marks are **77.93**.
- The `study_hours` column contains **5 missing values**.
- Study hours range from **5.01 to 8.99 hours**.
- Student marks range from **68.57 to 86.99**.

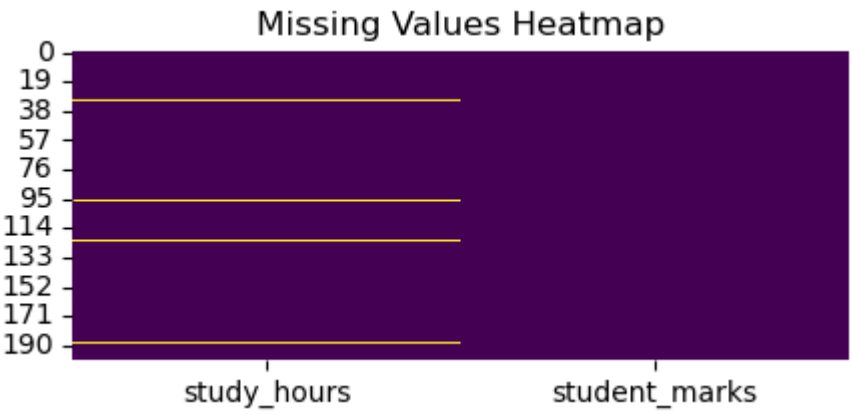
7. Check Missing Values

In [11]:
```
marks.isnull().sum()
```

Out[11]:
```
study_hours      5
student_marks    0
dtype: int64
```

Missing Values using Heatmap

In [12]:
```
plt.figure(figsize=(5,2))

sns.heatmap(marks.isnull(),
            cbar=False,
            cmap='viridis')

plt.title("Missing Values Heatmap")

plt.show()
```



Why use it?

- Yellow/light cells → Missing values (`NaN`)
- Purple/dark cells → Available values

This shows exactly **which rows** contain missing data.

The heatmap visualizes the location of missing values in the dataset.

Purpose

- Detects missing values visually.
- Identifies which rows and columns contain `NaN`.
- Helps decide the appropriate missing value treatment.

Missing Values Percentage

In [13]:
```
missing_percent = marks.isnull().mean()*100
print(missing_percent)
```

```
study_hours      2.5
student_marks    0.0
dtype: float64
```

not missing value percentage

In [14]:
```
no_missing_percent = marks.notnull().mean()*100
print(no_missing_percent)
```

```
study_hours       97.5
student_marks    100.0
dtype: float64
```

8. Handle Missing Values

This is a very common interview question.

Why do we use Mean to fill missing values?

We use the **mean (average)** because it provides a reasonable estimate for missing values in **continuous numerical data**, especially when the data is approximately normally distributed and there are no extreme outliers.

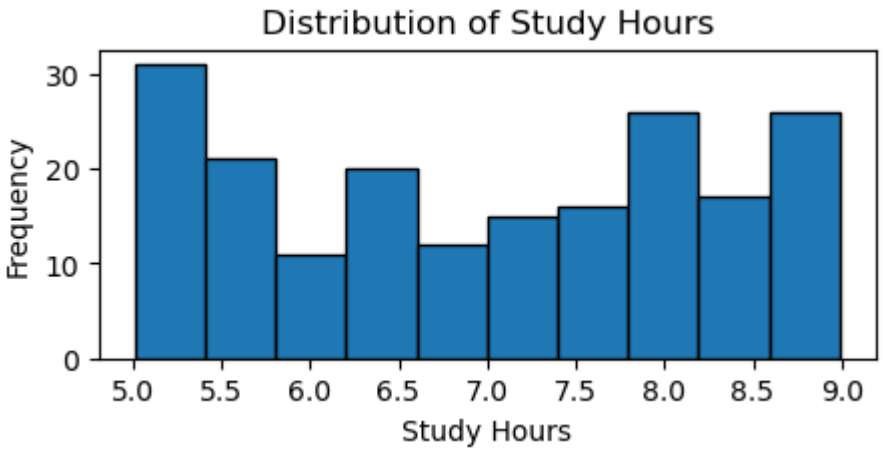
When to Use Mean, Median, and Mode

Method	Best Used For	Example
Mean	Continuous numerical data with no significant outliers	Age, Height, Weight, Study Hours, Marks
Median	Numerical data with outliers or skewed distribution	Salary, House Price, Income
Mode	Categorical or discrete data	Gender, City, Color, Blood Group

Before choosing Mean, you should check whether the data is approximately normally distributed and whether it contains outliers.

Histogram (Most Common)

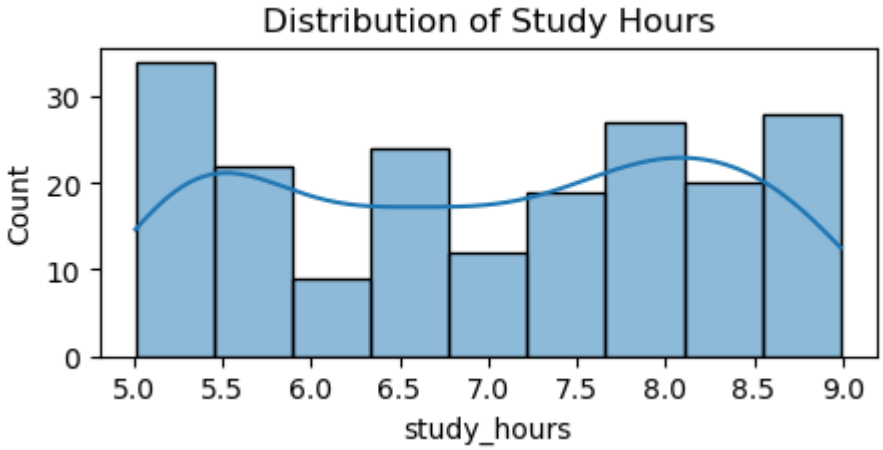
In [15]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(5,2))

plt.hist(marks["study_hours"], bins=10, edgecolor="black")

plt.title("Distribution of Study Hours")
plt.xlabel("Study Hours")
plt.ylabel("Frequency")

plt.show()
```



Distribution Plot (Histogram + KDE)

In [16]:
```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(5,2))
```

```
sns.histplot(marks["study_hours"], kde=True)

plt.title("Distribution of Study Hours")

plt.show()
```



1. Histogram

- The data is spread between **5 and 9 study hours**.
- The histogram is **not perfectly bell-shaped (normal)**.
- It does **not show strong left or right skewness**.

2. Histogram + KDE

- The KDE curve is **not a smooth bell curve**.
- It has **more than one peak (multimodal)**, so the distribution is not perfectly normal.
- However, it is **not heavily skewed**.

Can you directly say "Use Mean"?

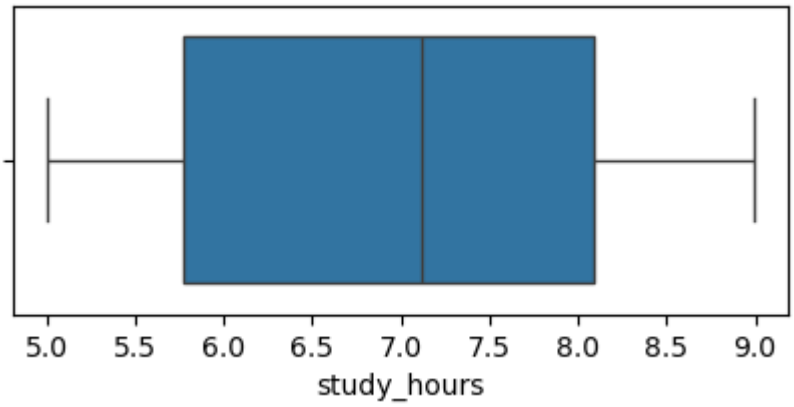
No. From these plots alone, you **cannot confidently conclude** that the mean is the best choice.

You should also check:

In [17]:
```
# Check skewness
print(marks["study_hours"].skew())
```

```
-0.06131042179501742
```

In [18]:
```
# Check outliers
plt.figure(figsize=(5,2))
sns.boxplot(x=marks["study_hours"])
plt.show()
```



Since:

- ✅ Skewness = **-0.0613** (approximately symmetric)
- ✅ No significant outliers
- ✅ Numerical column (`study_hours`)
- ✅ Only 5 missing values

Use Mean Imputation.

Use mean to fill missing handling

In [19]:
```
marks.mean()
```

Out[19]:
```
study_hours       6.995949
student_marks    77.933750
dtype: float64
```

In [20]:
```
# Data Cleaning
marks2 = marks.fillna(marks.mean())
```

In [21]:
```
#If your dataset has only one missing numerical column (study_hours), an even be
# marks2["study_hours"] = marks["study_hours"].fillna(marks["study_hours"].mean(
```

In [22]:
```
## Verify Missing Values
marks2.isnull().sum()
```

Out[22]:
```
study_hours      0
student_marks    0
dtype: int64
```

The `isnull().sum()` function is used to check the number of missing values in each column after data preprocessing.

Observation

- `study_hours` contains **0 missing values**.
- `student_marks` contains **0 missing values**.

Conclusion

All missing values have been successfully handled, and the dataset is now complete and ready for further preprocessing and machine learning model training.

9.Prepare the data for Machine Learning algorithms

In [23]:
```
marks # original dataset
```

Out[23]:

	study_hours	student_marks
0	6.83	78.50
1	6.56	76.74
2	NaN	78.68
3	5.67	71.82
4	8.67	84.19
...	...	...
195	7.53	81.67
196	8.56	84.68
197	8.94	86.75
198	6.60	78.05
199	8.35	83.50

200 rows × 2 columns

In [24]:
```
marks2 # clean dataset
```

Out[24]:

	study_hours	student_marks
0	6.830000	78.50
1	6.560000	76.74
2	6.995949	78.68
3	5.670000	71.82
4	8.670000	84.19
...	...	...
195	7.530000	81.67
196	8.560000	84.68
197	8.940000	86.75
198	6.600000	78.05
199	8.350000	83.50

200 rows × 2 columns

In [25]:
```
marks2.isnull().sum()
```

Out[25]:
```
study_hours      0
student_marks    0
dtype: int64
```

In [26]:
```
marks2.head()
```

Out[26]:

	study_hours	student_marks
0	6.830000	78.50
1	6.560000	76.74
2	6.995949	78.68
3	5.670000	71.82
4	8.670000	84.19

In [27]:
```
marks2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 2 columns):
 #   Column         Non-Null Count  Dtype  
---  ------         --------------  -----  
 0   study_hours    200 non-null    float64
 1   student_marks  200 non-null    float64
dtypes: float64(2)
memory usage: 3.3 KB
```

In [28]:
```
marks.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 2 columns):
 #   Column         Non-Null Count  Dtype  
---  ------         --------------  -----  
 0   study_hours    195 non-null    float64
 1   student_marks  200 non-null    float64
dtypes: float64(2)
memory usage: 3.3 KB
```

10. Analyze Relationship

Interview Answer

> How do you choose the independent and dependent variables?

Answer:

> First, I read the problem statement to identify the prediction goal. The variable I want to predict is the **dependent variable (Y)**. The variables used to make that prediction are the **independent variables (X)**. In the Student Marks dataset, the goal is to predict **student marks**, so `student_marks` is the dependent variable, while `study_hours` is the independent variable because it is used to predict the marks.

Purpose

- Understand the relationship between input and output variables.
- Identify positive, negative, or no correlation.
- Detect trends and outliers.
- Select a suitable machine learning model.

Scatter Plot

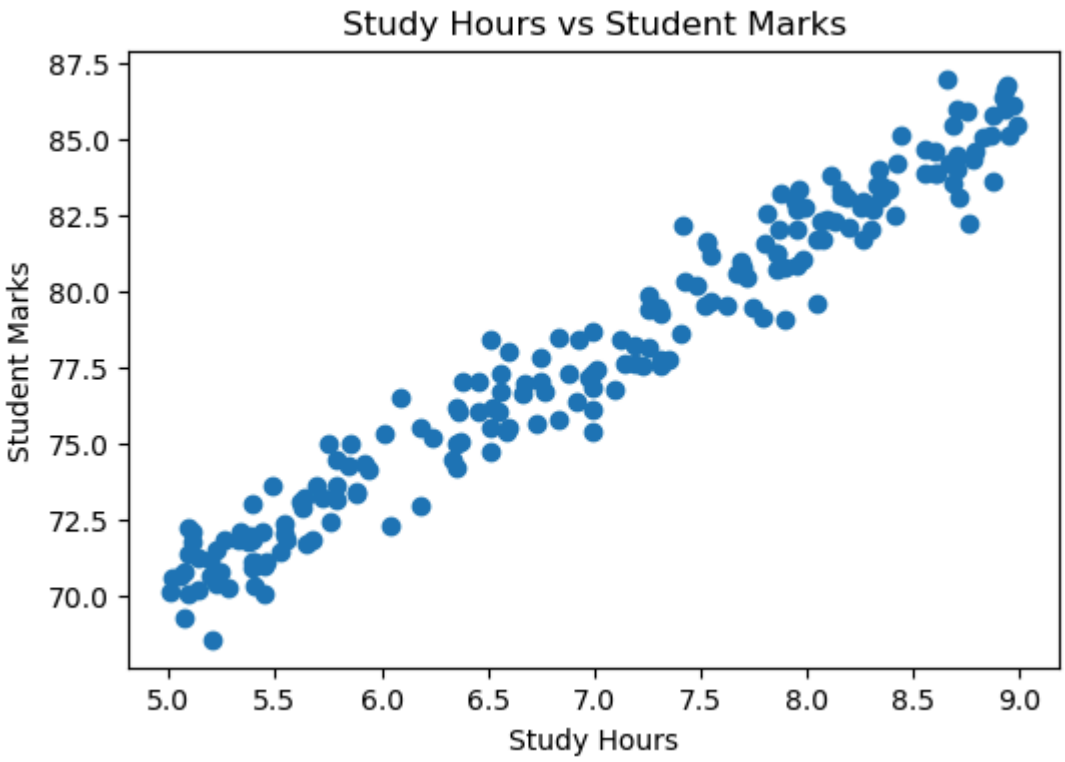
- **A scatter plot is used to visualize the relationship between two numerical variables.**
- **Each point represents one student's study hours and corresponding marks.**

Purpose

- Identify the relationship between `study_hours` and `student_marks`.
- Check whether the relationship is positive, negative, or absent.
- Detect outliers.
- Determine whether Linear Regression is suitable.

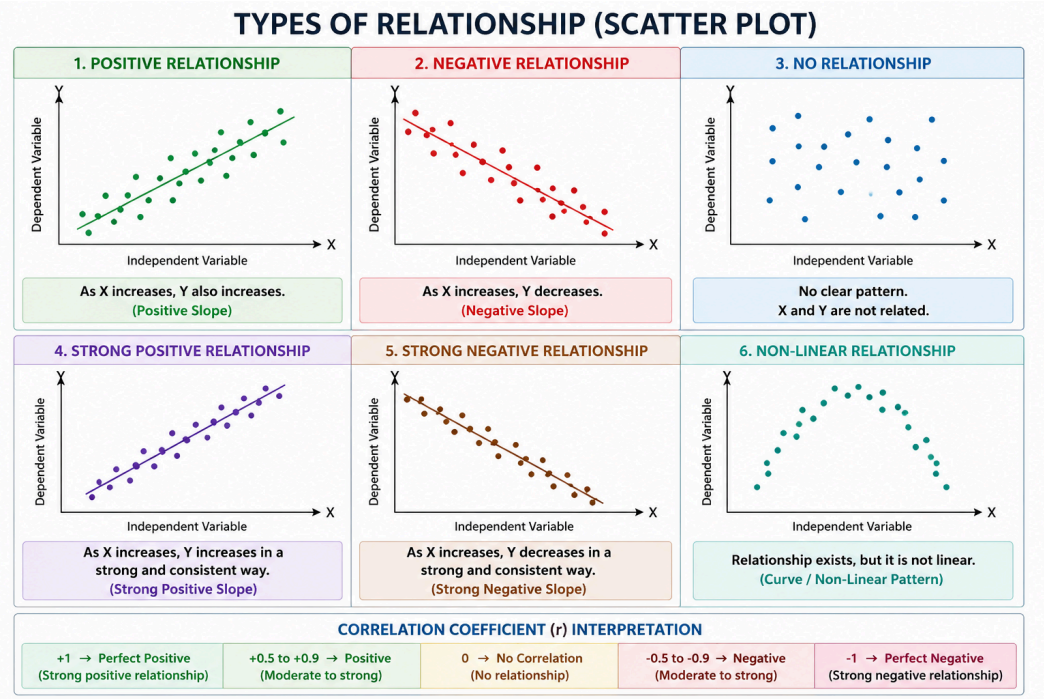
In [29]:
```
plt.figure(figsize=(6,4))

plt.scatter(marks2["study_hours"],
            marks2["student_marks"])

plt.title("Study Hours vs Student Marks")
plt.xlabel("Study Hours")
plt.ylabel("Student Marks")
```

```
plt.show()
```



Observation

- If the points form an upward trend, it indicates a **positive relationship**, meaning students who study more generally score higher marks.



!

Correlation plot

- Correlation measures the strength and direction of the relationship between numerical variables.

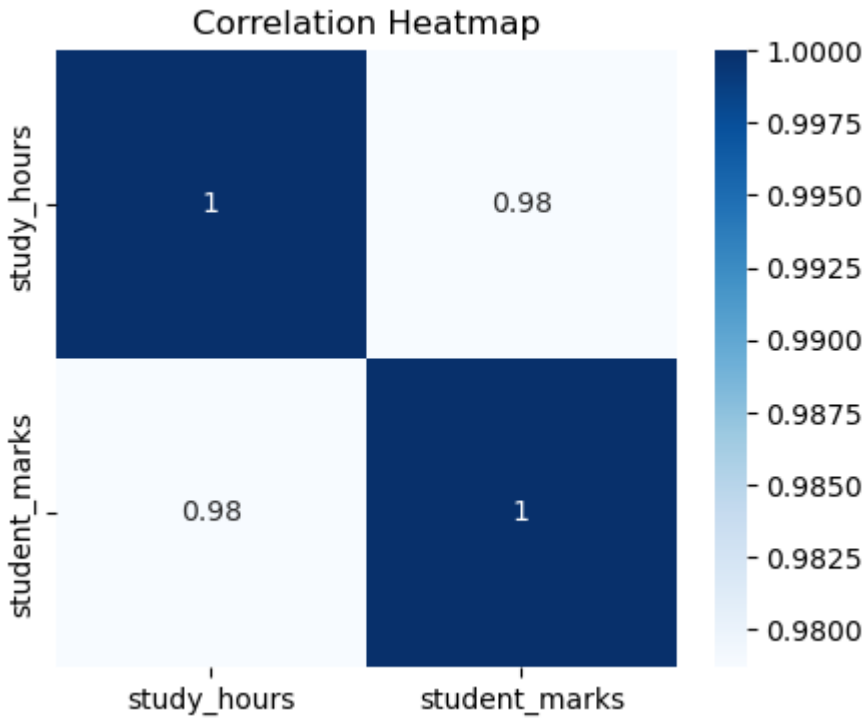
- The correlation coefficient ranges from **-1 to +1.**

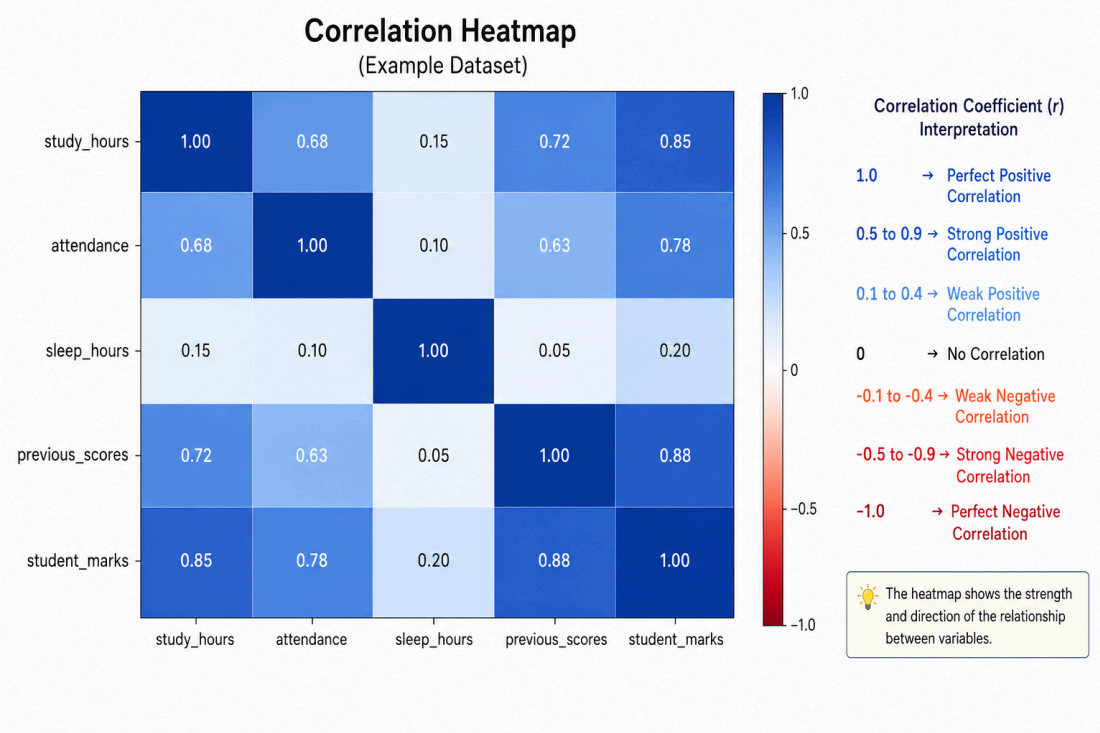
In [30]:
```
marks2.corr()
```

Out[30]:

	study_hours	student_marks
study_hours	1.000000	0.978696
student_marks	0.978696	1.000000

In [31]:
```
# correlation Heatmap
plt.figure(figsize=(5,4))

sns.heatmap(marks2.corr(),
            annot=True,
            cmap="Blues")

plt.title("Correlation Heatmap")

plt.show()
```





!

- The **scatter plot** shows an upward linear trend.
- The **correlation coefficient** is close to **+1**.

Then:

✅ There is a **strong positive relationship** between `study_hours` and `student_marks`, making **Linear Regression** an appropriate model for predicting student marks.

11. Separate Features (X) and Target (Y). (split dataset)

The dataset is divided into **input features (X)** and the **target variable (Y)**.

- **Features (X):** Independent variables used to make predictions.
- **Target (Y):** Dependent variable that the model learns to predict.

For this dataset:

- `study_hours` is the target(y).
- `student_marks` is the feature (x).## Separate Features and Target

In [32]:
```
# Separate the independent variable (Feature - X)
# Drop the target column 'student_marks'
X = marks2.drop("student_marks", axis="columns")

# Separate the dependent variable (Target - Y)
# Drop the feature column 'study_hours'
y = marks2.drop("study_hours", axis="columns")

# Display the shape of X and y
```

```
print("Shape of X =", X.shape)
print("Shape of y =", y.shape)
```

```
Shape of X = (200, 1)
Shape of y = (200, 1)
```

OR

Independent Variable (Feature) X = mark2[["study_hours"]] # Dependent Variable (Target) y = mark2["student_marks"] # Display the shape of X and y print("Shape of X =", X.shape) print("Shape of y =", y.shape)

In [33]:
```
X
```

Out[33]:

	study_hours
0	6.830000
1	6.560000
2	6.995949
3	5.670000
4	8.670000
...	...
195	7.530000
196	8.560000
197	8.940000
198	6.600000
199	8.350000

200 rows × 1 columns

In [34]:
```
y
```

Out[34]:

	student_marks
0	78.50
1	76.74
2	78.68
3	71.82
4	84.19
...	...
195	81.67
196	84.68
197	86.75
198	78.05
199	83.50

200 rows × 1 columns

12. Train-Test Split

Split the Dataset

The dataset is divided into **training** and **testing** sets.

- **Training Set:** Used to train the machine learning model.
- **Testing Set:** Used to evaluate the model's performance on unseen data.

A common split is **80% training** and **20% testing**.

In [35]:
```
# Import the train_test_split function
from sklearn.model_selection import train_test_split

# Split the dataset into training and testing sets
# 80% of the data is used for training and 20% for testing
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    train_size=0.8,
    test_size=0.2,
    random_state=0
)

# Display the shape of the training and testing datasets
print("Shape of X_train =", X_train.shape)
print("Shape of y_train =", y_train.shape)
print("Shape of X_test  =", X_test.shape)
print("Shape of y_test  =", y_test.shape)
```

```
Shape of X_train = (160, 1)
Shape of y_train = (160, 1)
Shape of X_test  = (40, 1)
Shape of y_test  = (40, 1)
```

In [36]:
```
X_train
```

Out[36]:

	study_hours
134	6.51
66	7.86
26	6.51
113	7.95
168	7.95
...	...
67	8.26
192	8.71
117	8.83
47	5.01
172	7.35

160 rows × 1 columns

In [37]:
```
X_test.shape
```

Out[37]:
```
(40, 1)
```

In [38]:
```
y_train
```

Out[38]:

	student_marks
134	78.39
66	81.25
26	74.75
113	80.86
168	82.68
...	...
67	81.70
192	84.03
117	85.04
47	70.11
172	77.78

160 rows × 1 columns

In [39]:
```
y_test
```

Out[39]:

	student_marks
18	82.50
170	71.18
107	73.25
98	83.64
177	73.64
182	86.99
5	81.18
146	82.75
12	79.50
152	81.70
61	79.41
125	85.95
180	77.19
154	78.45
80	84.00
7	85.46
33	84.35
130	73.19
37	78.21
74	77.59
183	83.87
145	85.15
45	72.96
159	80.72
60	73.61
123	79.53
179	78.17
185	79.63
122	76.83
44	82.38
16	76.04
55	85.48
150	71.87

	student_marks
111	75.04
22	70.67
189	79.87
129	74.49
4	84.19
83	75.36
106	72.10

Train-Test Split

The dataset is divided into **training** and **testing** sets using the `train_test_split()` function.

Purpose

- **Training Set (`X_train`, `y_train`)**: Used to train the machine learning model.
- **Testing Set (`X_test`, `y_test`)**: Used to evaluate the model's performance on unseen data.

Parameters

- **test_size = 0.2** → 20% of the data is used for testing, and 80% is used for training.
- **random_state = 0** → Ensures that the data is split in the same way every time the code is executed, making the results reproducible.

Output

- **X_train** → Training feature data.
- **y_train** → Training target data.
- **X_test** → Testing feature data.
- **y_test** → Testing target data.

Random State

The `random_state` parameter controls the randomness of the train-test split.

Purpose

- Ensures the dataset is split in the same way every time the code is executed.
- Makes the experiment reproducible.
- Helps compare different machine learning models using the same training and testing data.

Note

The values `0` and `42` are commonly used seed values. They do not affect the quality of the model; they only determine the random split of the dataset.

Interview Answer

Question: Why do we use `random_state=42`?

Answer:

> `random_state` is used to make the train-test split reproducible. It initializes the random number generator with a fixed seed, so every execution produces the same training and testing sets. The value `42` is simply a commonly used seed; any integer such as `0`, `10`, or `100` can also be used. The choice of seed does not improve model accuracy—it only controls reproducibility.

13. Select Model and Train the Linear Regression Model

The Linear Regression model is trained using the training dataset (`X_train` and `y_train`).

During training, the model learns the relationship between the input feature (`study_hours`) and the target variable (`student_marks`). It calculates the best-fit line that minimizes the prediction error.

Purpose

- Learn the relationship between study hours and student marks.
- Estimate the best-fit regression line.
- Prepare the model for making predictions on new data.

In [40]:
```
# Linear Regression Equation:
# y = m * x + c

# Import the Linear Regression algorithm
from sklearn.linear_model import LinearRegression   #LinearRegression() is a cla


# Create an object (instance) of the Linear Regression model
lr = LinearRegression()
lr   # lr is an object (instance) of that class.
```

Out[40]:
```
▾ LinearRegression
▶ Parameters
```

y = m * x + c

Where:

- **y** = Predicted value (Student Marks)
- **m** = Slope (Coefficient)
- **x** = Independent variable (Study Hours)
- **c** = Intercept (Constant)

What is fit()?

fit() is a method (function) inside the LinearRegression class.

It trains the model using the training data.

What is `fit()`?

Definition:

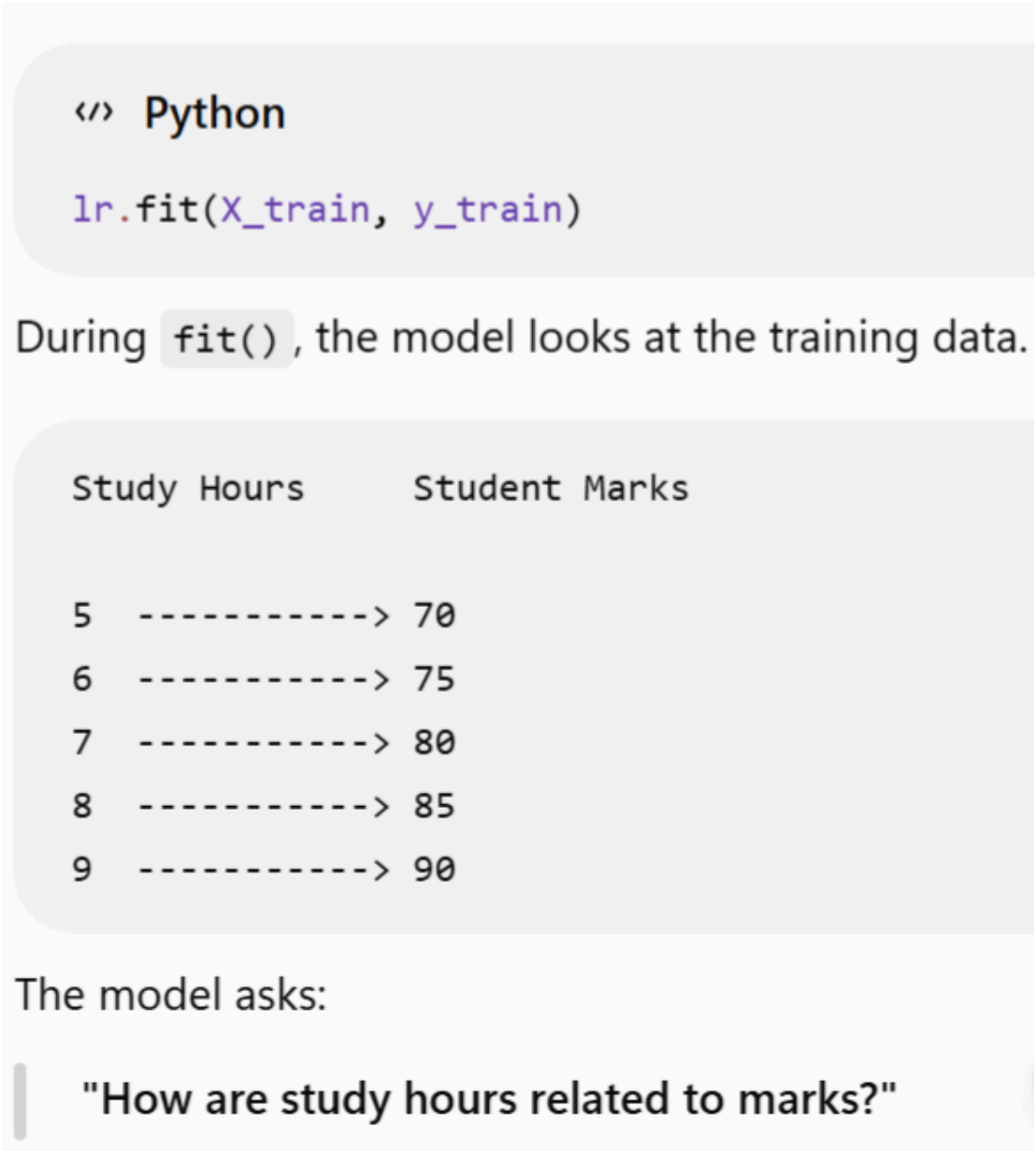
> The `fit()` method trains the machine learning model using the training data. During training, the model learns the relationship between the input (`X`) and output (`y`) and calculates the best-fit equation.

```
from sklearn.linear_model import LinearRegression

lr = LinearRegression()
```

At this stage:

```
LinearRegression
        |
        ▼
Model Created
(No Knowledge)
```

The model has **not learned anything** yet.



In [41]:
```
lr.fit(X_train,y_train)
```

Out[41]:
```
LinearRegression
▶ Parameters
```

In [42]:
```
lr.get_params()
```

Out[42]:
```
{'copy_X': True,
 'fit_intercept': True,
 'n_jobs': None,
 'positive': False,
 'tol': 1e-06}
```

Model Parameters

The `get_params()` method returns all the configuration parameters (hyperparameters) of the Linear Regression model.

Purpose

- Display the model's parameter settings.
- Check the default values.
- Verify any custom parameters before training the model.

Note: `get_params()` shows only the model configuration. It does **not** display the learned coefficient or intercept.

Meaning of Each Parameter

Parameter	Default Value	Explanation
`copy_X`	`True`	Creates a copy of the input data so the original dataset remains unchanged.
`fit_intercept`	`True`	Calculates the intercept (`c`) in the equation **y = mx + c**. If `False`, the line is forced to pass through the origin (0,0).
`n_jobs`	`None`	Number of CPU cores used for computation. `None` means the default behavior.
`positive`	`False`	If `True`, the model forces all coefficients (slopes) to be positive.
`tol`	`1e-06`	Tolerance used to determine when the optimization should stop.

In [ ]:

In [43]:
```
#m = Slope (Coefficient)
lr.coef_
```

Out[43]:
```
array([[3.93037294]])
```

In [44]:
```
 #C=Intercept (Constant)
lr.intercept_
```

Out[44]:
```
array([50.45063632])
```

In [25]:
```
m = 3.93
c = 50.44
y  = m * 10 + c
y
```

Out[25]:
```
89.74000000000001
```

If a student studies 10 hours, the Linear Regression model predicts that the student will score approximately 89.74 marks.

In [45]:
```
m = 3.93
c = 50.44
y  = m * 11 + c
y
```

Out[45]:
```
93.67
```

```
lr.predict(X)
```

Where:

- `lr` → Trained Linear Regression model.
- `predict()` → Predicts the output (`y`) for the given input (`X`).

In [47]:
```
# Predict marks for 10 study hours
lr.predict([[8]]).round(2)
```

```
C:\Users\kunal\anaconda\Lib\site-packages\sklearn\utils\validation.py:2749: UserW
arning: X does not have valid feature names, but LinearRegression was fitted with
feature names
  warnings.warn(
```

Out[47]:
```
array([[81.89]])
```

In [48]:
```
lr.predict([[11]]).round(2)
```

```
C:\Users\kunal\anaconda\Lib\site-packages\sklearn\utils\validation.py:2749: UserW
arning: X does not have valid feature names, but LinearRegression was fitted with
feature names
  warnings.warn(
```

Out[48]:
```
array([[93.68]])
```

In [52]:
```
y_pred  = lr.predict(X_test)
y_pred
```

Out[52]:
```
array([[83.50507271],
       [70.84927186],
       [72.93236952],
       [85.35234799],
       [73.20749562],
       [84.48766595],
       [80.12495199],
       [81.85431608],
       [80.91102657],
       [82.20804964],
       [78.98514384],
       [84.84139951],
       [77.84533568],
       [77.68812077],
       [83.22994661],
       [85.78468901],
       [84.9593107 ],
       [72.61793968],
       [78.71001773],
       [79.18166248],
       [84.2911473 ],
       [85.6274741 ],
       [74.74034107],
       [81.3433676 ],
       [72.02838374],
       [80.40007809],
       [78.98514384],
       [82.09013845],
       [77.94732382],
       [82.24735337],
       [75.44780819],
       [84.60557713],
       [71.63534645],
       [75.48711192],
       [70.29901965],
       [78.98514384],
       [75.32989701],
       [84.52696967],
       [74.07217767],
       [71.4388278 ]])
```

OR

- using loop(no need here)

In [51]:
```
m = 3.93
c = 50.44

while True:
    x = float(input("Enter Study Hours (0 to Exit): "))

    if x == 0:
        break

    y = m * x + c
    print(f"Predicted Marks = {y:.2f}")
```

```
Predicted Marks = 81.88
Predicted Marks = 85.81
Predicted Marks = 89.74
```

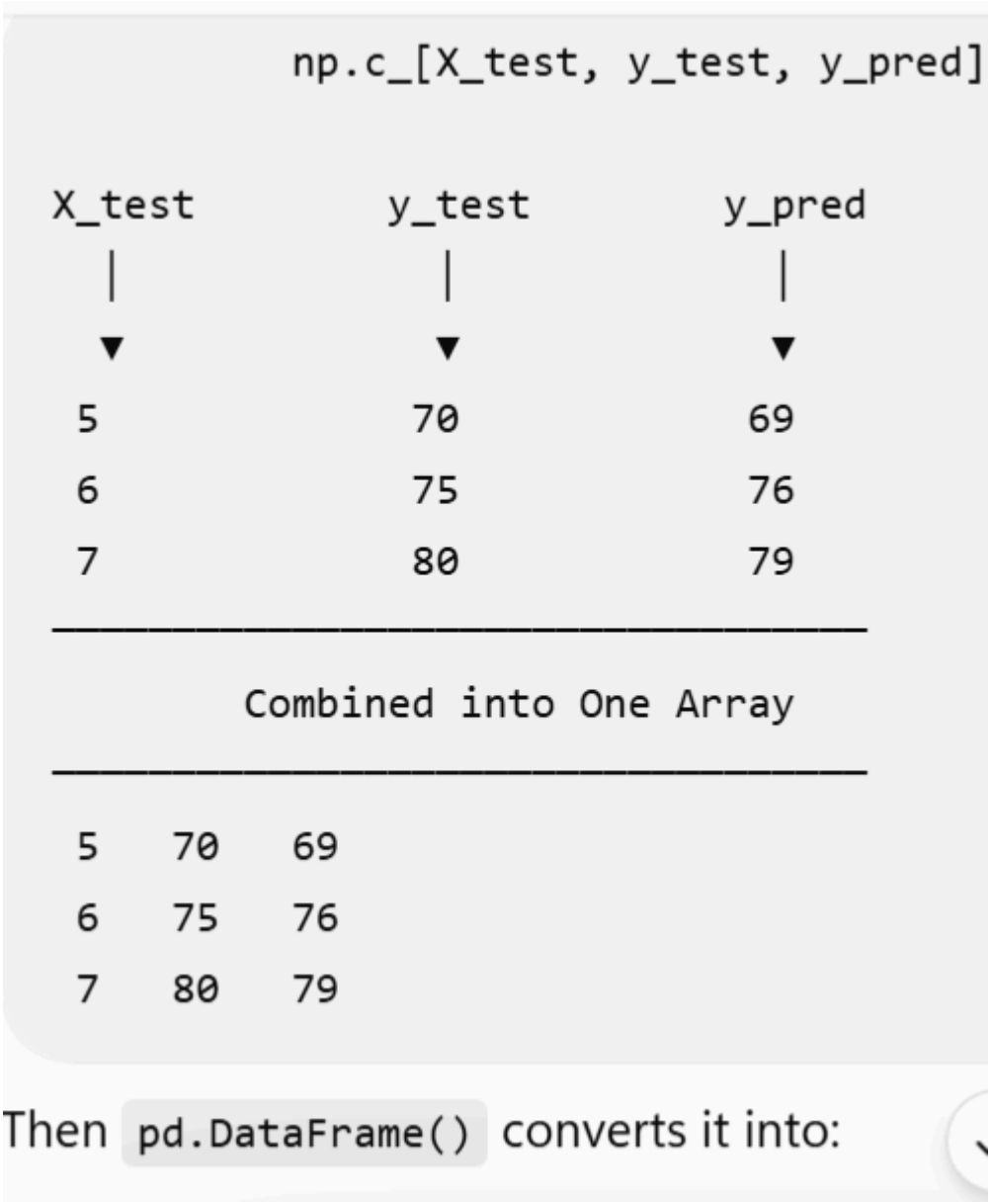
```
Predicted Marks = 93.67
```

In [29]:
```
# Create a DataFrame to compare actual and predicted values

pd.DataFrame(
    np.c_[X_test, y_test, y_pred],
    columns=[
        "study_hours",
        "student_marks_original",
        "student_marks_predicted"
    ]
)
#np.c_[] combines multiple arrays column-wise into a single NumPy array
```

Out[29]:

	study_hours	student_marks_original	student_marks_predicted
0	8.410000	82.50	83.505073
1	5.190000	71.18	70.849272
2	5.720000	73.25	72.932370
3	8.880000	83.64	85.352348
4	5.790000	73.64	73.207496
5	8.660000	86.99	84.487666
6	7.550000	81.18	80.124952
7	7.990000	82.75	81.854316
8	7.750000	79.50	80.911027
9	8.080000	81.70	82.208050
10	7.260000	79.41	78.985144
11	8.750000	85.95	84.841400
12	6.970000	77.19	77.845336
13	6.930000	78.45	77.688121
14	8.340000	84.00	83.229947
15	8.990000	85.46	85.784689
16	8.780000	84.35	84.959311
17	5.640000	73.19	72.617940
18	7.190000	78.21	78.710018
19	7.310000	77.59	79.181662
20	8.610000	83.87	84.291147
21	8.950000	85.15	85.627474
22	6.180000	72.96	74.740341
23	7.860000	80.72	81.343368
24	5.490000	73.61	72.028384
25	7.620000	79.53	80.400078
26	7.260000	78.17	78.985144
27	8.050000	79.63	82.090138
28	6.995949	76.83	77.947324
29	8.090000	82.38	82.247353
30	6.360000	76.04	75.447808
31	8.690000	85.48	84.605577
32	5.390000	71.87	71.635346

	study_hours	student_marks_original	student_marks_predicted
33	6.370000	75.04	75.487112
34	5.050000	70.67	70.299020
35	7.260000	79.87	78.985144
36	6.330000	74.49	75.329897
37	8.670000	84.19	84.526970
38	6.010000	75.36	74.072178
39	5.340000	72.10	71.438828



study_hours	student_marks_original	student_marks_predicted
5	70	69
6	75	76
7	80	79

Model Evaluation

After making predictions, the model is evaluated using regression evaluation metrics. These metrics measure how close the predicted values are to the actual values.

Most Important Regression Metrics

- MAE (Mean Absolute Error)
- MSE(Mean Squared Error)

- RMSE(Root Mean Squared Error)
- R^2 Score(Coefficient of Determination)

Purpose

- Measure the prediction error.
- Evaluate the model's performance.
- Compare different regression models.
- Determine the accuracy of the trained model.

In [54]:
```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

import numpy as np

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
print("R²  :", r2)
```

```
MAE : 0.8574561538746253
MSE : 1.0443968573561397
RMSE: 1.021957365723316
R²  : 0.9521841793508594
```

Model Evaluation Summary

The Linear Regression model was evaluated using MAE, MSE, RMSE, and R^2 Score.

Results

- **MAE:** 0.8575
- **MSE:** 1.0444
- **RMSE:** 1.0220
- **R^2 Score:** 0.9522

Interpretation

- The **MAE** indicates that the model's predictions differ from the actual student marks by **less than one mark on average**.
- The **MSE** is low, showing that large prediction errors are minimal.
- The **RMSE** indicates that the typical prediction error is approximately **one mark**, making it easy to interpret because it is measured in the same unit as the target variable.
- The **R^2 Score of 0.9522** means the model explains approximately **95.22% of the variation** in student marks.

Conclusion

The evaluation metrics demonstrate that the Linear Regression model performs very well on this dataset. The prediction errors are low, the R^2 Score is high, and the model generalizes well to unseen data. Therefore, the model is considered **satisfactory**, and additional fine-tuning is not required.

Model Generalization Check

In [55]:
```
lr
```

Out[55]:
```
▾ LinearRegression
▶ Parameters
```

In [56]:
```
# R² Score on the training dataset
train_score = lr.score(X_train, y_train)

# R² Score on the testing dataset
test_score = lr.score(X_test, y_test)

print("Training R²:", train_score)
print("Testing R² :", test_score)
```

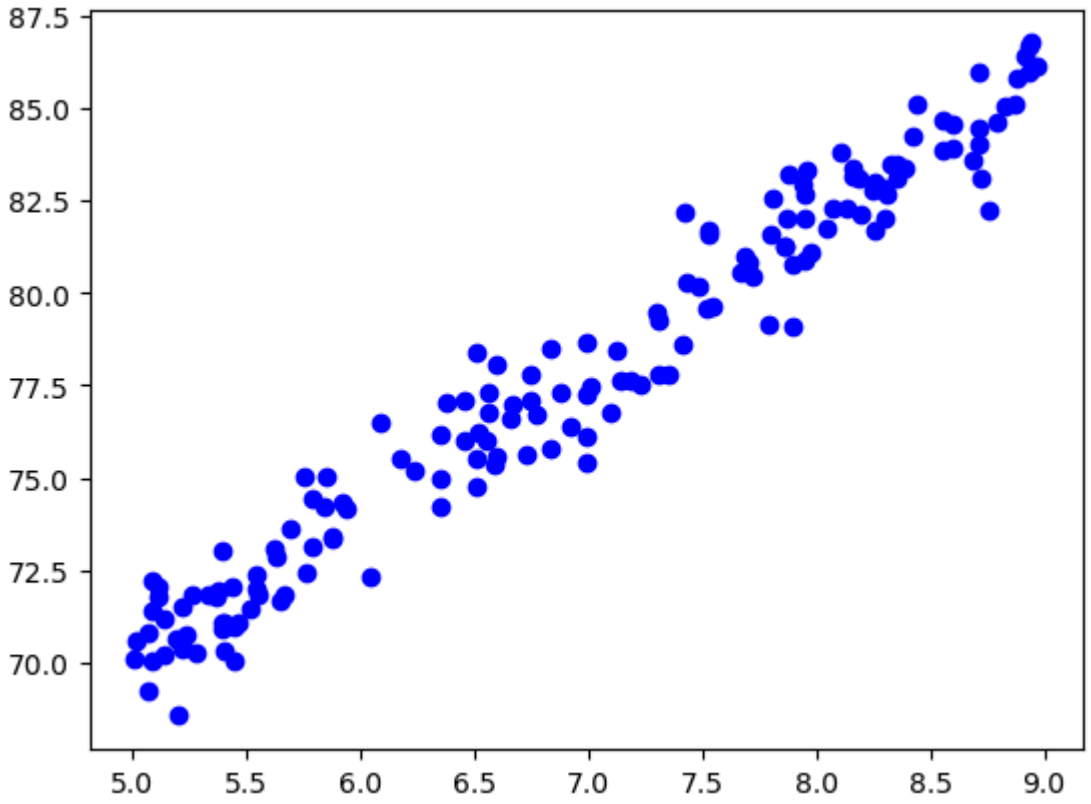
```
Training R²: 0.9584528455152638
Testing R² : 0.9521841793508594
```

In [ ]:
```
Training R² ≈ Testing R² → Low bias, low variance (good model).
Training R² >> Testing R² → High variance (overfitting).
Training R² and Testing R² both low → High bias (underfitting).
```

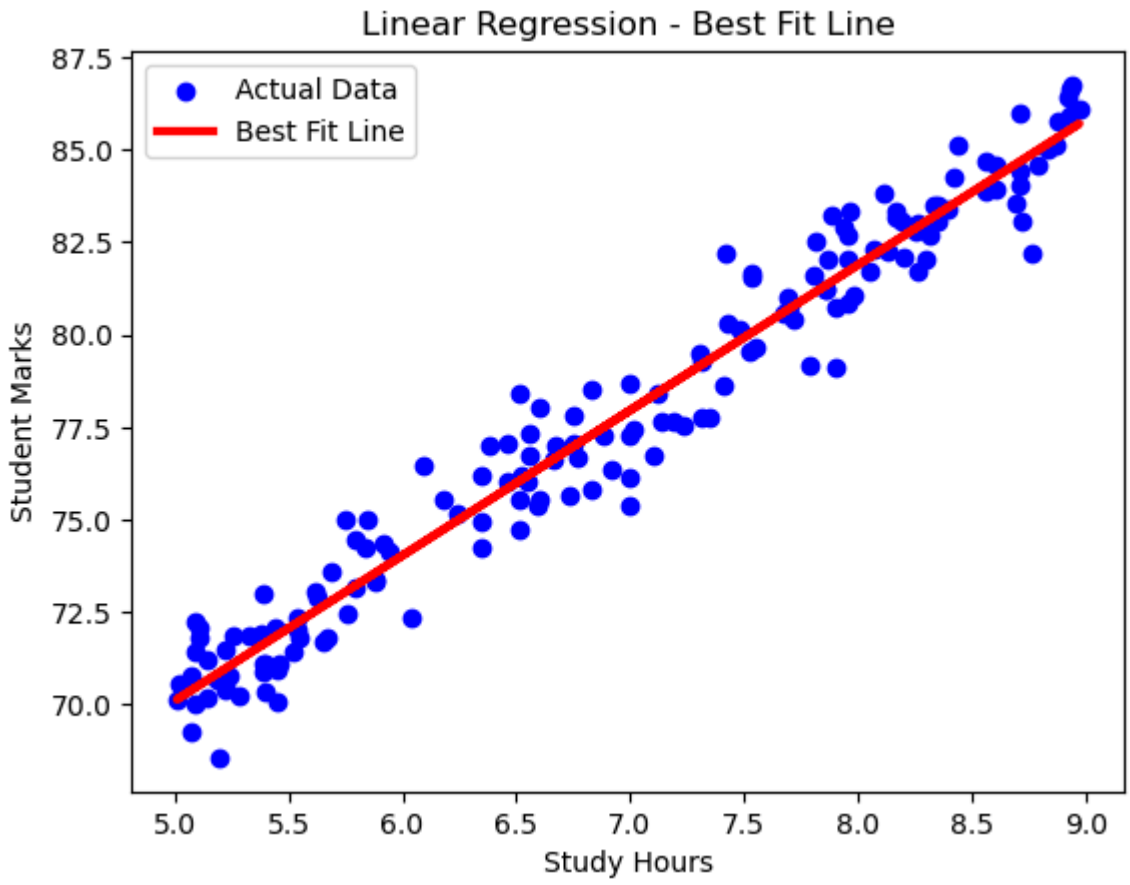
The model has **low bias** and **low variance**, making it a **good and satisfactory model**.

Training Data Visualization

In [68]:
```
# Plot training data
plt.scatter(X_train, y_train, color="blue")
plt.show()
```



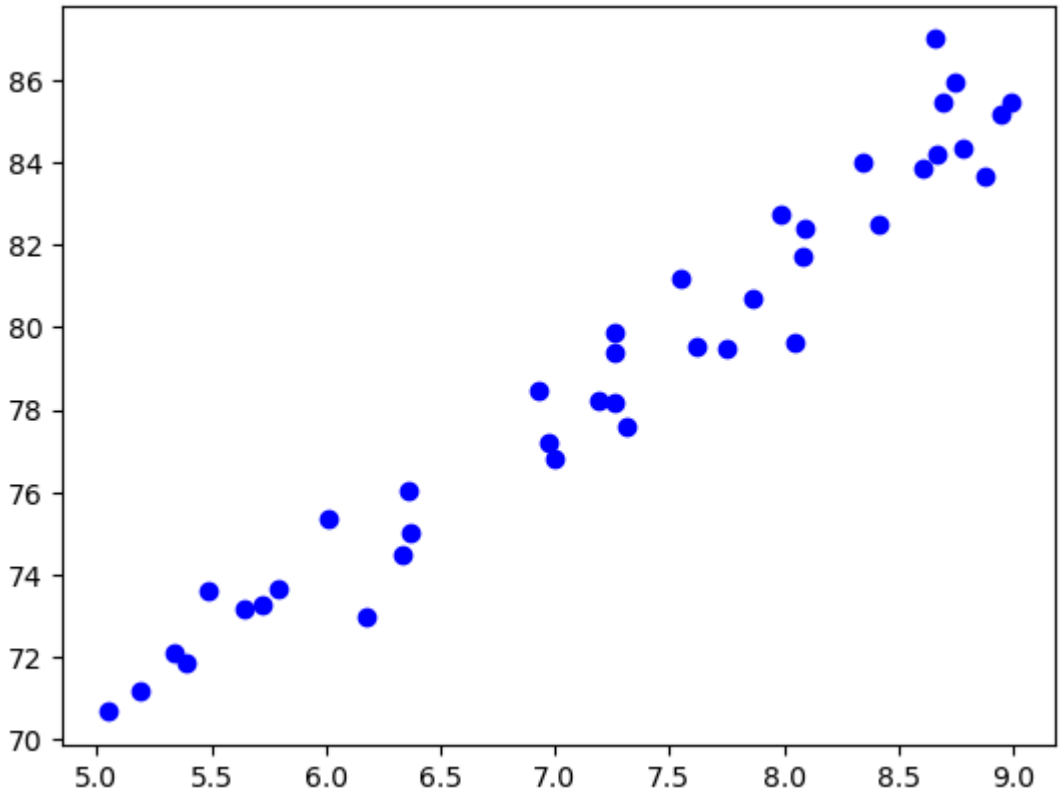
In [64]:
```
# Scatter plot of actual training data
plt.scatter(X_train, y_train, color="blue", label="Actual Data")

# Best Fit Line
plt.plot(X_train, lr.predict(X_train), color="red", label="Best Fit Line",linewi

plt.xlabel("Study Hours")
plt.ylabel("Student Marks")
plt.title("Linear Regression - Best Fit Line")
plt.legend()

plt.show()
```



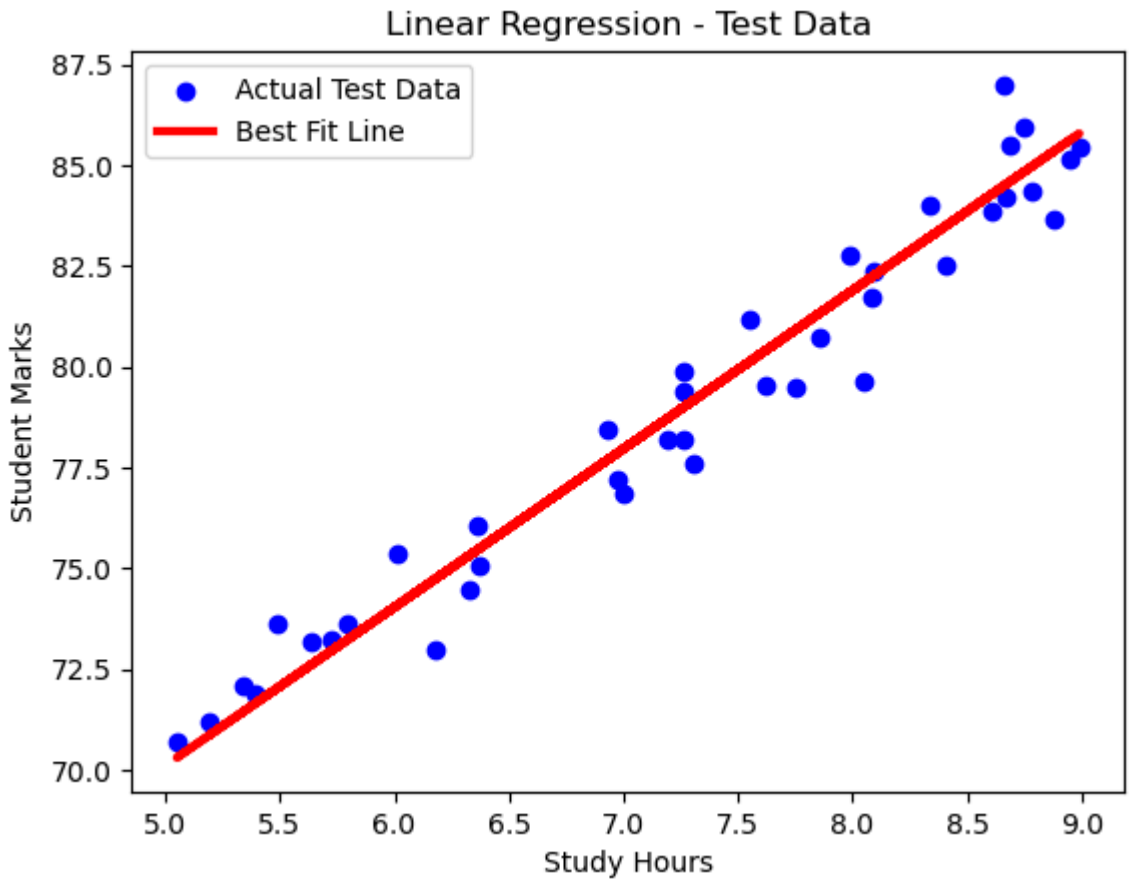
Testing Data Visualization

In [65]:
```
# Scatter plot of actual test data
plt.scatter(X_test, y_test, color="blue", label="Actual Test Data")
```

Out[65]:
```
<matplotlib.collections.PathCollection at 0x2a2269cad50>
```



In [67]:
```
# Scatter plot of actual test data
plt.scatter(X_test, y_test, color="blue", label="Actual Test Data")

# Best Fit Line for test data
plt.plot(X_test, lr.predict(X_test), color="red", linewidth=3, label="Best Fit L

plt.xlabel("Study Hours")
plt.ylabel("Student Marks")
plt.title("Linear Regression - Test Data")
plt.legend()

plt.show()
```



Save ML Model

In [70]:
```
lr
```

Out[70]:

LinearRegression

▶ Parameters

In [71]:
```
# Step 1: Import Joblib
import joblib

# Step 2: Save the trained model
joblib.dump(lr, "student_marks_model.pkl")

print("Model saved successfully.")
```

```
Model saved successfully.
```

In [72]:
```
pwd
```

Out[72]:
```
'C:\\Users\\kunal\\Downloads'
```

In [73]:
```
# Step 3: Check current working directory
import os

print(os.getcwd())
```

```
C:\Users\kunal\Downloads
```

In [74]:
```
# Step 4: Load the saved model
model = joblib.load("student_marks_model.pkl")

print("Model loaded successfully.")
```

```
Model loaded successfully.
```

In [77]:
```
import pandas as pd


# New input data
new_data = pd.DataFrame({
    "study_hours": [9]
})

# Prediction
prediction = model.predict(new_data)

print("Predicted Marks:", round(prediction[0][0], 2))          #prediction[0][0]
```

```
Predicted Marks: 85.82
```

In [ ]:
```

```

In [80]:
```
print(lr.predict([[9]])[0][0].round(2))
```

```
85.82
C:\Users\kunal\anaconda\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
```

Deploy the Model

You can deploy using:

- Streamlit (best for beginners)
- Flask
- FastAPI

Launch, monitor, and maintain your system

In [1]:
```
__name__
```

Out[1]:
```
'__main__'
```

Student Marks Dataset

Objective

The dataset is used to predict student marks based on study hours using Machine Learning Regression.

Columns

1. **study_hours**
    - Independent Variable (X)
    - Represents the number of hours a student studies.
    - Data Type: Float
2. **student_marks**
    - Dependent Variable (Y)
    - Represents the marks scored by the student.
    - Data Type: Float

Dataset Characteristics

- Total Columns: 2
- Input Feature: study_hours
- Target Feature: student_marks
- Data Type: Numerical
- Machine Learning Type: Supervised Learning
- Algorithm Type: Regression

Missing Values

The `study_hours` column contains `NaN`, which represents missing data. These missing values should be handled before training the model.

Goal

Build a regression model that predicts student marks based on study hours.

In [ ]: